

2026-10-01

valer: a documentation linter for R packages

Nic Crane
Independent R consultant

Executive Summary

R has good tooling for keeping code consistent, but nothing equivalent for the prose in a package: the help pages, vignettes, READMEs and comments. Hard-to-read documentation costs users time and puts off newcomers, and there's more of it about as more package code and documentation gets written by AI tools.

[Vale](#) is the prose linter most other ecosystems have settled on. Its rules are defined as data, there's a library of published styles, and it already integrates with editors, GitHub Actions and pre-commit. It can read Markdown and pull the text out of comments in R files, but it doesn't know anything else about R. Its default styles weren't written with R packages in mind, and using it at all means installing and configuring a binary by hand.

This proposal is for a Vale style for R package documentation, and an R package, [valer](#), that installs Vale, writes the config for a project, and runs it from R. The style picks out the rules from Vale's existing library that make sense for package docs, with thresholds set by running them over real CRAN packages and a vocabulary of R terms, so that the first run on a real package reports problems rather than noise. The package also writes a block into a project's `AGENTS.md` so that AI coding agents run the linter on the documentation they write.

The deliverables are the style, published for anyone using Vale directly, and the package on CRAN, over two milestones ending in February 2027, for \$3,600.

Signatories

Project team

- [Nic Crane](#): R developer with over 10 years of experience, one of the maintainers of the [arrow](#) R package and contributor to [ellmer](#).

Consulted

- [Etienne Bacher](#): independent developer, author of [jarl](#), a fast linter for R funded by the ISC in 2025. Was consulted on an earlier draft of this as a more complex project which helped me revise it down to a smaller initial scope.

The Problem

R already has conventions and tooling for code style. [lintr](#) ([Hester et al. 2025](#)), [air](#) and [jarl](#) are widely used, and it's normal for a package to enforce them on CI, but there's nothing similar for documentation. `R CMD check` confirms that documentation exists and matches the function signatures, but whether a human can actually read it is left to authors and reviewers. That applies to the help pages, vignettes and READMEs users see as much as to the comments contributors rely on.

Documentation that's hard to read, whether because of long dense sentences or a tone that assumes things are obvious, increases cognitive load ([Sweller 1988](#)), frustrates users, and puts people off using a package at all. rOpenSci's [software review process](#) asks reviewers to judge whether documentation is clear and sufficient, but reviewers are doing that by hand. More packages are also being written with the help of, or entirely by, AI tools, and without something checking, the result is often verbose.

There's been some prior work, including my own. The [docreview](#) package was on CRAN from 2021 to 2024 with five checks including readability scores, and stalled because its architecture was more complicated than it needed to be. The [spelling](#) package checks words but not prose. Outside R, [Vale](#) is a mature prose linter with packaged style guides that runs in editors and on CI. It handles Markdown and can extract comment text from R files, but it knows nothing about how an R package is laid out or what R vocabulary looks like, and you have to install and configure a separate binary to use it.

What's missing is an opinionated documentation linter for R packages that's usable from R and helps both humans and AI tools write better docs.

The proposal

Overview

The proposal is a Vale style for R package documentation, plus an R package, [valer](#), that installs Vale, configures it and runs it, so documentation can be linted from R, from the command line or on CI. It covers a package's vignettes and READMEs and the comments in its R source files, and reports the file, line, rule and a message explaining what was wrong. For R that's a data frame, and for other tools, it's Vale's own JSON output.

Vale already has rules for sentence length and readability scores, spread across several styles and tuned for general prose. This project would pick out the subset that makes sense for package documentation, set thresholds by running them over real packages so the defaults aren't noisy, and add a vocabulary of common R terms. These become the default style, which ships with the package and is also published for people who use Vale directly. Rules are configured from a Vale config file at the package root, which the package can generate, so people can turn things off or tighten them to taste.

Vale handles extracting comments from R files, parsing Markdown, running rules and integrating with editors, and because its rules are data rather than code, people can contribute rules without

writing any R. It also has an official GitHub Action and a pre-commit hook, so once the style is installable those work without any extra code, and the package documents how to use them. This means there isn't much new code to write.

The package itself has three parts: finding an existing Vale binary or downloading one for the user's platform when asked, the way tinytex does; a setup function that writes the Vale config for a project and installs the style; and a function that runs Vale and returns lints as a data frame. The setup function also writes a short block into the project's AGENTS.md telling AI coding agents to run the linter on any documentation they write and fix what it reports. This means agents are checked against the same rules as everyone else.

The main dependency is Vale itself, which is actively maintained and publishes binaries for Linux, macOS and Windows on every release. There are no ongoing costs.

Project plan

Start-up phase

The project will be developed in a public GitHub repository under the MIT licence, with issues used to track all work.

Technical delivery

I've given Vale a trial run over an existing R package to test out comment extraction. The results were noisy but workable. It picks up comments from R source files with correct line numbers and handles vignettes and READMEs without any extra work.

Milestone 1 - 31st January 2027: working package and first rules

This milestone starts with talking to package maintainers and reading through documentation feedback in public software review threads, such as rOpenSci's, to decide which rules are most useful in the default style. The code delivered is the R package that installs or locates Vale, runs it over a package and returns lints as a data frame, with an initial set of rules and snapshot tests, working on Linux, macOS and Windows.

Milestone 2 - 28th February 2027: default style and release

This milestone covers the remaining rules, the vocabulary of R terms, and the setup function that writes the Vale config and the AGENTS.md block, with each rule documented with examples of what passes and what fails. The style is run over the 100 most downloaded CRAN packages and the results used to tune the rules and defaults. The package goes to CRAN and the style is published so Vale users can install it directly, with documentation covering use from R, from the command line, and on CI via Vale's existing GitHub Action and pre-commit hook. The project is written up for the R Consortium blog.

Other aspects

All code is MIT licensed and developed in the open on GitHub. The package will go to CRAN and the style will be published on GitHub so it can be used with Vale directly. Publicity is a post on the R Consortium blog, posts on LinkedIn, Mastodon and Bluesky at each milestone, and a submission to R Weekly.

Budget & funding plan

The budget is labour only, at USD 90 per hour, for 40 hours of part-time work over two months. There are no other costs. GitHub covers hosting, CI and releases.

Milestone	Target	Hours	USD
1: Working package and first rules	31 Jan 2027	20	1,800
2: Default style and release	28 Feb 2027	20	1,800
Total		40	3,600

Success

Definition of done

The project is complete when:

- the package is published on CRAN and installs or locates Vale on Linux, macOS and Windows
- it lints R code comments, vignettes and READMEs using a default style that ships with the package, with rules configurable from a file at the package root
- the style is also published so that it can be installed and used directly by Vale users
- the defaults have been tuned so that the tool runs without error on the 100 most downloaded CRAN packages
- the setup function writes the Vale config and the AGENTS.md block for a project
- use from R, the command line, and CI via Vale's GitHub Action and pre-commit hook is documented

Measuring success

This is a success if a package author can go from nothing to a first set of lints in a couple of minutes, with a default style that reports real problems rather than noise, and if it becomes a normal part of checking a package, like lintr is. The first can be measured directly. The tuning run over the 100 most downloaded CRAN packages gives a hit rate per rule, and the defaults get set from that. The second shows up as packages adding the style to their CI, issues and pull requests from people outside the project, and people using the style with Vale without the R package.

Metric	Target	Measurement
Install and first lints on a fresh machine	Under 5 minutes following the README	Checked on CI on Linux, macOS and Windows
Default style on the 100 most downloaded CRAN packages	Runs without error on all 100	Tuning run, with results published in the repo
Package on CRAN, style installable by Vale	Both released	CRAN, Vale's Packages config
Blog post	1	R Consortium blog

Future work

- Roxygen-aware linting of help pages, so that tags are handled properly, rules can be applied per help topic and example code is not treated as prose
- Linting of error and warning messages
- Additional named styles, so that organisations with a house style can publish their own rule sets for R packages

Hester, Jim, Florent Angly, Michael Chirico, et al. 2025. “Static Code Analysis for R.” *Journal of Open Source Software* 10 (108): 7240. <https://doi.org/10.21105/joss.07240>.

Sweller, John. 1988. “Cognitive Load During Problem Solving: Effects on Learning.” *Cognitive Science* 12 (2): 257–85. https://doi.org/10.1207/s15516709cog1202_4.